Debouncing & Throttling

8.4 – Debouncing & Throttling

🎯 What Interviewers Actually Ask

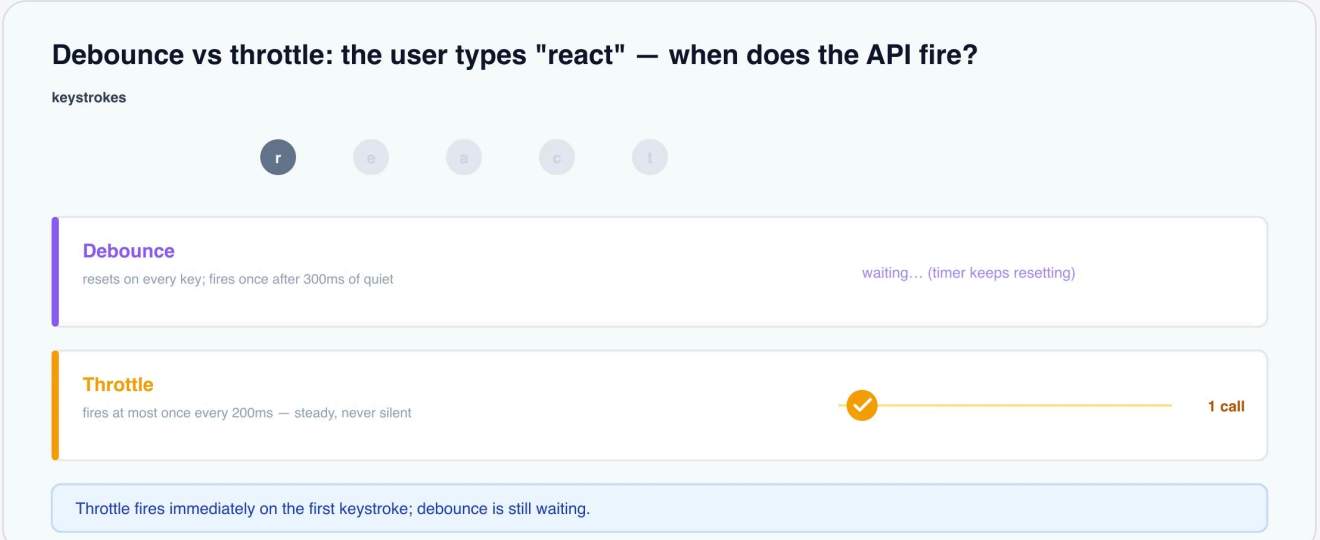
Question	Level	Frequency
"What is debouncing?"	🟡	★★★★★
"What is the difference between debounce and throttle?"	🟡	★★★★☆
"Implement a debounced search input"	🟡	★★★★☆

The Difference

Debounce: Waits until the user *stops* doing something for a set delay, then fires once. Like a search input that waits until the user stops typing before calling the API.

Throttle: Fires at most once every X milliseconds, regardless of how many events happen. Like a scroll handler that updates a progress bar at most 10 times per second.

	Debounce	Throttle
Fires when	User stops for X ms	At most once every X ms
Best for	Search input, form validation, resize	Scroll events, mouse tracking, rate-limited API
User types "react"	API fires once, 300ms after "t"	API fires multiple times during typing



useDebounce hook

jsx

JSX Copy

```jsx
function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);
```

```
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

Using it for search

jsx

JSX Copy

```jsx
function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const debouncedQuery = useDebounce(query, 300);

  useEffect(() => {
    if (!debouncedQuery.trim()) { setResults([]); return; }

    let cancelled = false;
    setLoading(true);

    fetch(`/api/search?q=${encodeURIComponent(debouncedQuery)}`)
      .then(r => r.json())
      .then(data => { if (!cancelled) { setResults(data); setLoading(false
      .catch(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, [debouncedQuery]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeh
      {loading && <p>Searching ... </p>}
      {!loading && results.length > 0 && (
        <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
      )}
      {!loading && debouncedQuery && results.length === 0 && <p>No results
```

useThrottle hook

jsx

JSX Copy

```jsx
function useThrottle(value, interval = 500) {
  const [throttledValue, setThrottledValue] = useState(value);
  const lastRun = useRef(Date.now());

  useEffect(() => {
    const now = Date.now();

    if (now - lastRun.current ≥ interval) {
      lastRun.current = now;
      setThrottledValue(value);
    } else {
      const timer = setTimeout(() => {
        lastRun.current = Date.now();
        setThrottledValue(value);
      }, interval - (now - lastRun.current));

      return () => clearTimeout(timer);
    }
  }, [value, interval]);

  return throttledValue;
}
```

Common Mistakes

Mistake 1: Debouncing the setter instead of the value. Debounce the *value* and use it as a useEffect dependency. Don't try to debounce `setQuery` directly.

Mistake 2: Not cancelling the previous request. Without a `cancelled` flag or AbortController, old API results can overwrite newer ones (race condition).

Mistake 3: Using throttle where debounce is needed. Throttle for scroll/resize (continuous events that need periodic updates). Debounce for search input (want the final value only).

🗣️ Interview Answer

> "Debouncing delays execution until the user stops performing an action for a set time, like waiting for the user to finish typing before calling a search API. Throttling limits execution to once per interval, useful for scroll or resize events. I implement both as custom hooks: useDebounce returns a debounced value that I use as a useEffect dependency, and useThrottle does the same but fires at a maximum rate. The key is always to cancel in-flight requests when a new one starts, using a cancelled flag or AbortController."

📚 Module 8 Complete!

#	Article	Core Concept
8.1	fetch vs axios vs React Query	HTTP clients, AbortController, caching, interceptors
8.2	Loading, Error & Empty States	State cascade, skeleton screens, retry, empty UX

8.4	Debouncing & Throttling	Custom hooks, search optimization, race conditions

QUICK CHECK — no marks, just for you.

1. A user types the word "react" into a search box. If the change handler is throttled rather than debounced, what happens to the search API?

- ◯ The API fires exactly once, after the user stops typing for the interval
- ◯ The API never fires until the user blurs the input
- ◯ The API fires once per keystroke, exactly as if no throttle were applied
- ◯ The API fires multiple times during typing, at most once per interval

Check answer

2. In the SearchBar example, the search effect sets `let cancelled = false` and returns a cleanup that sets it to true. What problem does this specifically prevent?

- ◯ A slower earlier request resolving late and overwriting the results of a newer request
- ◯ The debounce timer from firing after the component unmounts
- ◯ The fetch call from being sent at all when the query is empty
- ◯ React from rendering the input twice in StrictMode

Check answer

3. A teammate debounces a search input and reaches for `useDebounce`. According to the lesson, what should they actually debounce?

- ○ The fetch call, by wrapping it in a setTimeout inside onChange
- ○ The query value, then use the debounced value as a useEffect dependency
- ○ The setQuery state setter, so state updates less often
- ○ The onChange handler itself, wrapping it in useDebounce

Check answer

‹ Previous
Authentication & JWT in React

Next ›
Testing Fundamentals: What, Why, and How

LEARN MORE

ARTICLES & DOCS

- Debouncing & Throttling explained
  css-tricks.com
- Lodash — debounce & throttle
  lodash.com

VIDEOS



Debounce & throttle in 16 minutes
YouTube



Throttling & debouncing with examples
YouTube





React.js Interview Kit

Hand-picked references for this lesson.